Ace the Salesforce FDE Final Round — Playbook

Candidate: Shaikat Majumdar · **Format:** 45 min, 2:1 panel (leader + FDE), remote · **Project:** `salesforce-agentforce-enterprise-bridge`

0. Integrity rules (read first)

- This repo is a **reference implementation**. Say so in the first 30 seconds. Panels probe; invented production claims collapse under follow-ups.
- The "4 hours → 45 seconds / 70%" figures in your idea notes are **illustrative targets, not measurements**. Use only (a) numbers you **measured** (`make bench`), (b) numbers from **real past engagements you can defend** (e.g., your résumé's 99.2% task-completion, <5-day MVPs), or © an explicit **model with stated assumptions** (§6).
- Never claim a Salesforce capability you have not used. Say "I built the integration pattern against a Developer Org; I haven't operated Data Cloud at customer scale."

1. Competency → proof map

Competency	Proof in the project	Résumé tie-in
Influence stakeholders	SRS persona table; ambiguity/uncertainty split; autonomy tiers negotiated with compliance	Held the room with architects at J.P. Morgan; exec sponsors at Balyasny
Drive innovation & AI readiness	LLM planner w/ deterministic fallback; eval-as-tests; injection containment	Prompt debugging → 99.2% completion
Agility	Offline demo → live org in minutes; MVP in days	PoCs in <5 days
Build & deploy technical capabilities	100% coverage, CI matrix, Docker, SFDX deploy workflow	Hardened field tools into EKS/AKS services, ADRs
Integration platform & ecosystem	OAuth client creds, REST/SOQL, External Services, Platform-Event-ready, Kafka seam	REST/gRPC/webhooks/Kafka, ServiceNow/Archer/Jira
AI ecosystems & frameworks	Agentforce actions, Anthropic API, LangGraph-style planner/tool split	LangChain/LangGraph/LlamaIndex, Agentforce Specialist

2. Round-1 feedback → what to do

1. **Ambiguity vs uncertainty** → say the words aloud: *"That part was ambiguity—we resolved it by asking; this part was uncertainty—we resolved it by measuring."* (§5)
2. **Tie outcomes to ROI** → every technical slide ends with a *so-what* line (§6).
3. **Agentforce in forward-deployed work** → §7 cheat-sheet; show the manifest and External Service path.

3. Run of show (15-minute presentation + 30 min discussion)

Min	Slide	Say	Show
0:00–1:30	1 Who I am	17 yrs; FDE at Balyasny/J.P. Morgan/Millburn; agentic AI in regulated enterprises; travel-ready 25–40%	résumé one-liner
1:30–3:00	2 Problem & sponsor	Fragmented silos → agents hallucinate/leak/overreach. Sponsor ask: "resolve disputes automatically"	SRS table (ambiguity)
3:00–4:30	3 Discovery	Autonomy tiers: who may act alone vs needs human; uncertainty spikes (429s, drift)	decision-flow diagram
4:30–6:30	4 Architecture	Ground → plan → govern → act → audit; Salesforce mapping (§7)	context diagram
6:30–11:00	5 **Live demo**	Terminal runbook §4	terminal + Swagger + org
11:00–12:30	6 Hard problems	Drift, prompt injection, SF outage, duplicates	resilience table
12:30–14:00	7 Impact & ROI	measured + modeled	§6 table
14:00–15:00	8 Productionising	MVP vs scale; roadmap	improvements list

4. Terminal runbook (rehearse twice; have screenshots as fallback)

```
# T-30 min: pre-flight
git clone https://github.com/sm2774us/salesforce-agentforce-enterprise-bridge && cd salesforce-agentforce-enterprise-bridge
python3.12 -m venv .venv && source .venv/bin/activate && make install
make test                        # "Required test coverage of 100% reached"
set -a; source .env; set +a      # SF_LOGIN_URL / SF_CLIENT_ID / SF_CLIENT_SECRET (never on screen!)

# On stage
make demo                        # offline: PII stripped, ACME Corp. == Acme Corporation, $25k held for approval
bridge manifest | head -40       # Agentforce action catalog + JSON schema + requiresApproval
bridge demo --live               # same flow, real Cases in the Dev Org → switch to browser, show Case list + Bridge_Audit_Log__c
BRIDGE_API_KEYS='ak=agent:AGENT,pk=jane:APPROVER' bridge serve &    # then:
curl -s -XPOST localhost:8080/v1/events?wait=true -H 'X-API-Key: ak' -H 'content-type: application/json' \
  -d '{"source":"erp","account_name":"Globex Ltd","severity":"high","message":"Invoice mismatch","amount_usd":25000}'
curl -s localhost:8080/v1/approvals -H 'X-API-Key: pk'                        # pending item
curl -s -XPOST localhost:8080/v1/approvals/<ID>/decision -H 'X-API-Key: pk' -H 'content-type: application/json' -d '{"approve":true}'
curl -s -XPOST localhost:8080/v1/events -H 'X-API-Key: ak' -d '{"bogus":1}'  # 422 schema drift → DLQ
bridge bench -n 2000             # measured throughput (offline CRM)
```

Demo narration beats: (1) "Email and card number never leave the bridge unredacted." (2) "Three spellings, one account." (3) "Agent proposes; Control Tower disposes—$25k waits for a different human." (4) "Every decision is a row in `Bridge_Audit_Log__c`." (5) "Kill the network → breaker opens; `/readyz` flips to 503; audit outbox replays."

If live org fails: say so calmly, run `make demo`, show screenshot; failing gracefully *is* the resilience story.

5. Ambiguity vs uncertainty — scripts

- **Ambiguity (people can answer it):** *"'Resolve disputes automatically' had three readings—auto-close, auto-refund, auto-triage. I ran a 90-minute workshop with the COO, compliance and support leads, classified each decision by reversibility and dollar impact, and produced autonomy tiers. That is why `create_case` is LOW and `create_financial_review` is HIGH."*

- **Uncertainty (only experiments answer it):** *"Whether upstream schemas would drift and how Salesforce rate limits behaved weren't knowable by asking. I time-boxed spikes: a strict contract with* `extra=forbid` *+ DLQ, and fault injection against the client. Result: drift is caught at the edge; 429/5xx are retried with jitter, and the breaker caps blast radius."*
- **Framing sentence for any requirement:** "Is the unknown in someone's head (ambiguity) or in the world (uncertainty)? That decides workshop vs spike."

6. Business value & ROI (model, not claim)

Formula: `Savings/yr = alerts/yr × (manual_min − assisted_min)/60 × loaded_$/hr − run_cost`.
Illustrative inputs (state them as assumptions): 20,000 alerts/yr, 25 min manual triage, 3 min human-in-loop for high-risk (~10% of alerts), ~0 for low-risk, $85/hr → manual ≈ 8,333 h; assisted ≈ 20,000×(0.1×3 min)/60 ≈ 100 h + review; savings on the order of **$600–700k/yr** before run cost. Say: *"This is the sizing model I'd validate in week one with the customer's real alert volume."*
Measured, defendable: pipeline p50 ≈ 0.4 ms / p95 ≈ 0.5 ms and ~2.5k events/s offline (excludes CRM network); 0 PII in audit payloads (tested); 100% coverage; MVP from clone to live org in under an hour.
Also quantify **risk avoided**: HITL on 100% of financial actions; full audit trail for SOX-style review.

7. Agentforce / Salesforce forward-deployed cheat-sheet

- **Agentforce** = agents (Agent Builder/Script) + Topics + **Actions** (Flow, Apex, Prompt Templates, External Services/MCP) grounded by the **Einstein Trust Layer** (PII masking, zero data retention, audit). My bridge is the *external action plane*: OpenAPI → External Service → Flow action → agent Action.
- **Data Cloud:** ingest (streaming/batch/zero-copy), map to **DSO** → **DMO**, identity resolution → Unified Profile, vector search/RAG for grounding. Mapping: `TelemetryEvent → UnifiedEvent` (DMO), `EntityResolver` (identity resolution), `VectorIndex` (vector search), zero-copy = federate Snowflake/Databricks instead of copying.
- **Integration:** OAuth 2.0 (client credentials/JWT/web-server flows), Named/External Credentials, REST/Bulk/Composite APIs, Platform Events & Pub/Sub API (gRPC), CDC, **MuleSoft** (system/process/experience API layers) for heavy orchestration.
- **FDE loop:** discover → MVP in days → harden → enable customer team → feed gaps back to product. Know the **Deployment Strategist** partnership: they own the business case; you own the build.
- **Governance:** permission sets, FLS, run-as user, Shield/event monitoring, Einstein Trust Layer.
- **Say what you haven't done** and how you'd learn it fast.

8. Simulated Q&A

A. Project & architecture

Q1. Walk me through the project in one minute. A governed bridge: events are validated, de-duplicated, resolved to an account, PII-stripped, grounded with account-scoped context, planned (rules or LLM), then every tool call passes the Control Tower—RBAC, validation, risk gate, human approval for HIGH, execution against Salesforce, audit to `Bridge_Audit_Log__c`. Business outcome: automation of low-risk triage with provable control on high-risk actions.

Q2. Why not let Agentforce call Salesforce directly? It can and should for in-org data. The bridge handles what lives outside: heterogeneous sources, streaming, schema drift, enterprise-specific policy. It exposes those as governed Actions so the agent stays simple and the policy stays testable.

Q3. Why a deterministic planner by default? Auditable baseline, zero-cost, instant, and the fallback when the LLM is down or misbehaves. The LLM is an upgrade for ambiguous cases, never a single point of failure.

Q4. How does grounding reduce hallucination? The planner receives only redacted, account-scoped facts (counts, exposure, similar incidents), outputs must match registry names and JSON schemas, and invalid output is rejected twice (planner, Control Tower).

Q5. How does this map to Data Cloud? See §7 mapping; in production I'd replace my simulator with Streaming Ingestion/zero-copy and Data Cloud identity resolution and vector search, keeping the Action plane unchanged.

Q6. Why FastAPI/Python? Async I/O for connector-heavy workload, pydantic contracts, OpenAPI for free (that *is* the Agentforce External Service definition). For latency-critical paths I'd move hot loops to Rust/C++ (my background) behind the same contracts.

Q7. Why is the vector index per account? Tenant isolation: cross-account recall is a data-leak vector. Also bounds search cost to O(n log k) with $n \leq 100$.

Q8. Time/space complexity? Enqueue O(1); dedupe O(1) bounded LRU; entity resolution O(1) exact, O(K) fuzzy on miss; recall O(n log k); approvals O(1). All buffers bounded (queue, DLQ, audit, history).

B. Salesforce & Agentforce specifics

Q9. How does Agentforce invoke your service? Import `/openapi.json` as an External Service via a Named Credential, expose operations as Flow actions, add to the agent's topic as Actions with instructions describing when to use them. The manifest includes `requiresApproval` so topic instructions can say "tell the user it is

pending".

Q10. Why client-credentials OAuth? Server-to-server, no interactive user; scoped to a run-as integration user with the `Bridge_Integration` permission set. For production I'd consider JWT bearer with certificate rotation.

Q11. Where does the Einstein Trust Layer fit? It masks PII for in-platform prompts; my redaction protects the *external* path (my LLM calls, audit, CRM writes). Defence in depth, not duplication.

Q12. How would MuleSoft fit? Put system APIs in front of ERP/payments, process API for the triage orchestration, and let my bridge call Mule as the integration fabric when the customer already standardises on it —don't rebuild what they own.

Q13. Platform Events vs polling? Platform Events/Pub-Sub or CDC for SF→bridge notifications (e.g., Case approved in-org); avoids polling and API-limit burn.

Q14. How do you handle Salesforce API limits? Retry with full jitter on 429/5xx, breaker, bounded queue backpressure, batching via Composite/Bulk as next step, and monitoring daily API usage via `/limits`.

Q15. Human approval in Salesforce itself? Today approvals are API-driven; the roadmap is a Flow/Approval Process on Case with the bridge listening via Platform Event—so approvers work in their native UI.

C. Resilience, security, quality

Q16. What if Salesforce is down? Breaker opens after 5 failures, `/readyz` → 503, actions fail fast with FAILED audit entries, the audit outbox replays on recovery. Events remain bounded in queue; beyond capacity we return 429 so upstream retries.

Q17. Duplicate or out-of-order events? Dedupe by `event_id` (at-least-once safe). Ordering: key Kafka partitions by account in production.

Q18. Prompt injection? Event text is data inside delimiters; model output only proposes; allow-listed actions; schema validation; RBAC and HITL cannot be bypassed by text. Tested with disallowed-action and malformed outputs.

Q19. How do you protect secrets? Environment variables / GitHub Environment secrets with protection rules; masked repr; none in code or logs; Dependabot + CodeQL + pip-audit. Next: OIDC to cloud secret manager, short-lived creds.

Q20. What does 100% coverage prove? Every line/branch executed, not correctness. Correctness comes from behaviour tests (fault injection, adversarial LLM outputs, separation of duties). I treat coverage as a floor.

Q21. Biggest weakness today? In-memory approvals/audit outbox → single replica, lost on restart. First fix: Postgres + idempotent upsert on `Audit_Id__c`.

Q22. How do you evaluate the LLM planner? Golden dataset of events → expected action, run in CI nightly; metrics: action accuracy, invalid-output rate, fallback rate, approval-override rate from humans (the best signal).

Q23. Monitoring in production? Prometheus counters per decision, fallback/mirror failure counts, p95 latency, queue depth, DLQ size, audit pending sync; alerts on breaker open and DLQ growth.

D. Consultative / behavioural

Q24. Sponsor asks for "fully autonomous". Response? Reframe to outcomes; show autonomy tiers; start with LOW-risk automation, measure override rate, expand autonomy by evidence.

Q25. Ambiguity vs uncertainty — example? §5.

Q26. A stakeholder disagrees with your architecture. Write an ADR with options/trade-offs, run a time-boxed spike if the dispute is empirical, escalate only after data. (J.P. Morgan: ADRs for audit-ready services.)

Q27. Customer's engineers resent the "outsider". Co-build workshops, pair on the first Action, hand over tests/runbook; success = they operate it without me (Balyasny enablement).

Q28. Scope creep mid-MVP? Maintain a "deviation log"; surface business deviations early with impact on timeline/risk, let the sponsor choose; architecture stays aligned via ADRs.

Q29. Tell me about a failure. (Prepare a real one. Structure: context, your mistake, detection, fix, systemic change.) Example frame: shipped an agent without an override-rate metric; added it, and made it a release gate.

Q30. How do you feed back to product? Structured field notes: friction, repro, frequency, workaround, business impact; weekly with product eng. Example gaps to cite from this build: Agentforce action retries/idempotency semantics, approval UX for external actions.

Q31. 25–40% travel OK? Yes (already ~35% at Balyasny/40% at J.P. Morgan).

Q32. MVP vs scale priorities? MVP: contract, one happy path, HITL, audit, observability. Scale: persistence, Kafka, multi-replica, rate limits, evals, DR, cost controls.

E. Curveballs

Q33. Design this for 10M events/day. ~115/s average, ~1k/s peak: Kafka partitioned by account, stateless consumers on K8s HPA, Redis/Postgres for approvals, Data Cloud streaming ingestion, batched Composite writes, backpressure, tiered planners (rules first, LLM on ambiguity only for cost).

Q34. Reduce LLM cost/latency? Rules-first routing, prompt caching, small model for classification, cache by normalised message hash, timeouts + fallback.

Q35. GDPR deletion? Keep raw PII out of stores by design; audit holds redacted payloads; key by pseudonymous account key; deletion = remove CRM records + purge by key.

Q36. Why not LangGraph here? The flow is a fixed DAG with one decision; a framework would add weight without value. For multi-step reasoning with tool loops I'd use LangGraph (as in my `agentic-platform`), keeping the same Control Tower.

Q37. What would you do in your first 30/60/90 days? 30: shadow Deployment Strategist, learn product + customer environments, ship one MVP. 60: lead a delivery pod, reusable Action templates. 90: own a customer outcome end-to-end, contribute field feedback and reusable assets.

9. Questions to ask them

1. How do FDEs and Deployment Strategists split discovery vs build on a typical Agentforce engagement?
2. What does a successful first 90 days look like on this team?
3. Which Agentforce/Data Cloud gaps do FDEs most often feed back to product?
4. How are customer outcomes measured after handover?

10. Final checklist

- [ ] Repo cloned fresh; `make test` green; `.env` loaded; org logged in; Case list tab open
- [ ] Screenshots of demo + diagrams as fallback; slides ≤8
- [ ] ROI model numbers memorised with assumptions
- [ ] Two real past stories prepared (one win with metric, one failure)
- [ ] Rehearsed 15 minutes aloud, timed, twice